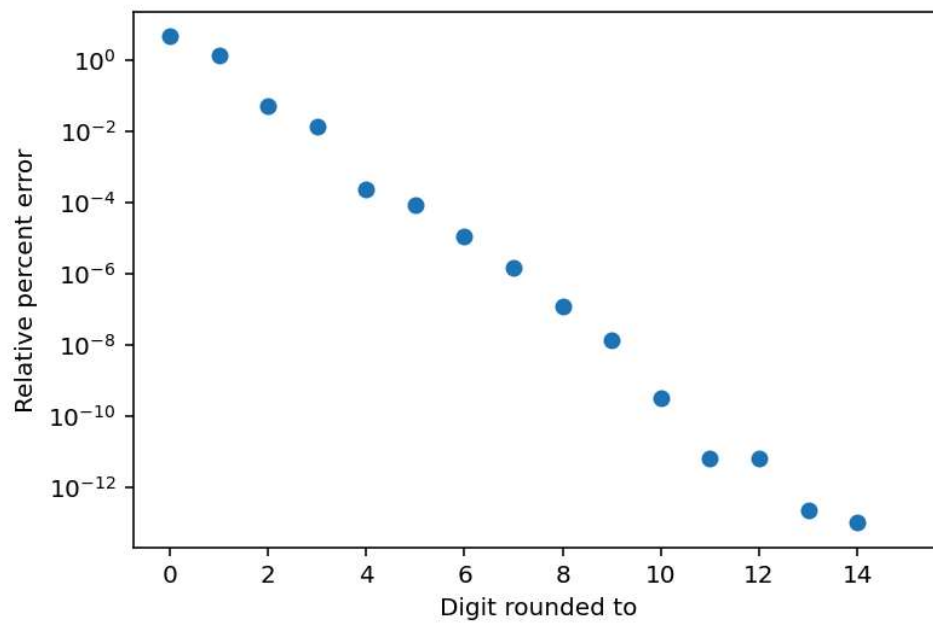
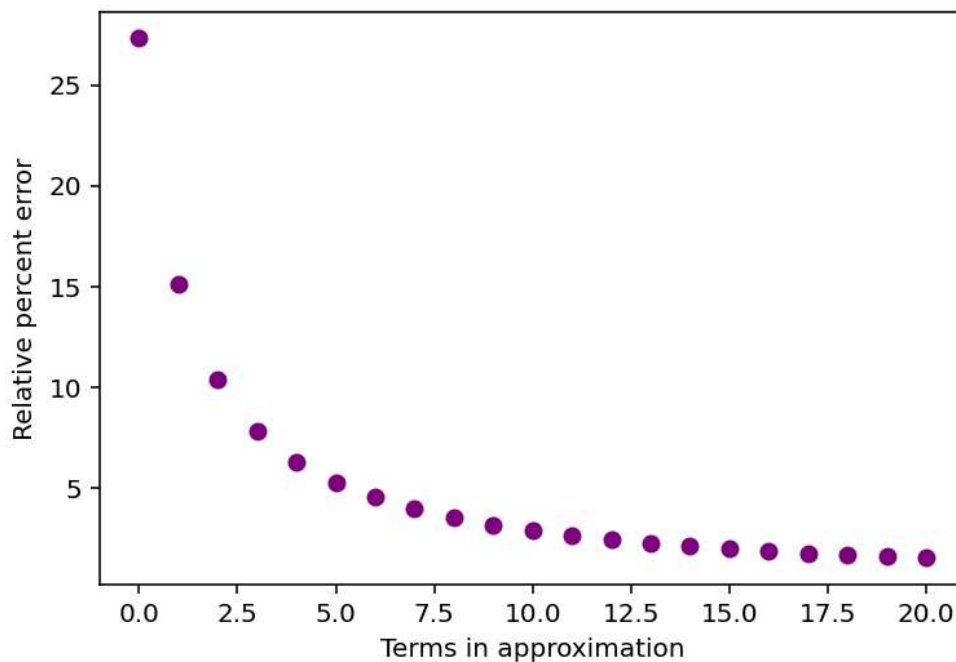




This figure shows an example of roundoff error, showing the relative percent error for the value of pi based off of how many digits the value is rounded to



This figure shows an example of truncation error, showing the relative percent error for the value of pi based off of how terms are used from the infinite series used to approximate pi

Truncation error comes from cutting off of terms in series that computers use to approximate values or functions. Rounding error comes from the computer cutting off digits in long values that it uses, making the final value less precise. Understanding these helps me to know when to expect a computer to be accurate, and when to expect slight levels of error, it also helps me to understand how computers do math in general. In this lab I learned how to more effectively model a series to provide estimates, such as Leibniz's approximation of π .

Step 1:

```
def roundoff_error():
    n = 15
    approx = np.zeros(n+1)

    for i in range(n+1):
        approx[i] = round(np.pi, i)
        #error = ( (np.pi-estimate) / np.pi)* 100

    error_array = abs(np.pi - approx) / np.pi * 100

    #Plot
    plt.figure()
    plt.scatter(np.arange(0,16,1), error_array)
    plt.yscale('log')
    plt.xlabel('Digit rounded to')
    plt.ylabel('Relative percent error')
```

Step 2:

```
def leibniz(N):
    pi_estimate = 0
    for k in range(N+1):
        pi_estimate += ((-1)**k) / (2*k +1)
    return 4 * pi_estimate

def truncation_error():
    N = 20
    approx_arr = np.zeros(N+1)

    for n in np.arange(0,N+1,1):
        approx_arr[n] = leibniz(n)

    error = abs(np.pi - approx_arr) / np.pi *100

    #Plot
    plt.figure()
    plt.scatter(np.arange(0,21,1), error, color = 'purple')
    plt.xlabel('Terms in approximation')
    plt.ylabel('Relative percent error')
```